

Methods Section:

We started with this dataset called `Transformed_TGSIM_Foggy_Bottom_200sec.csv`. It has 200 seconds of data on how different transportation modes move around in an urban intersection. There are autonomous vehicles, regular passenger cars that are not automated, and pedestrians, all tracked over time. Each entry is for one agent at a certain timestamp, and it includes stuff like a unique ID for the agent, the time in seconds, x and y positions in meters, speed in m/s, acceleration in m/s^2 , and what type of agent it is. We checked the data dictionary in `TGSIM_Data_Dictionary.xlsx` to make sure about the variables and units.

For processing everything, we used Python, version 3.10 or later, and pulled in libraries like pandas, numpy, matplotlib, and seaborn. Imported the CSV with `pandas.read_csv`. First thing, we cleaned it up by dropping rows with missing or bad values in important spots like ID, time, position, speed, and acceleration. We made sure data types were numbers where they should be. Then sorted by agent ID and time, so trajectories made sense in order. Got rid of any duplicates that had the same ID and timestamp, since that could mess up calculations later. Grouped agents into AVs, non-automated vehicles, and pedestrians using the type of variable.

To narrow it down to the intersection, we used polygon boundaries from that sample notebook, `Polygons.ipynb`. Applied to a point-in-polygon check to see if each point was inside or outside. Also, set up buffer zones around it to look at how agents act when approaching. That way, we could see adjustments in movement before they got in.

On behavioral metrics, we calculated some basics for comparisons. For speed profiles, mean, median, and standard deviation for each group, overall, and just at the intersection. Used the coefficient of variation, which is std dev over the mean, to check consistency. Acceleration helped with dynamics, and we marked deceleration events as anything under -1.0 m/s^2 . Counted by those per agent, average magnitude too, then grouped by type and location, inside or out.

It seems like interactions matter a lot here. So, we looked at the speed changes when entering the zone. For trajectories going in, subtract the entry speed from the lowest speed inside, as a way to spot caution or disruption. Also checked trajectory consistency with the standard deviation of speed and acceleration over time per agent. Lower variability means more predictability.

Visualization helped make sense of it. We made 2D plots of trajectories, x vs y, colored by agent type, with boundaries shown. Histograms and kernel density for speed and acceleration distributions using seaborn. Boxplots to compare metrics like speed, acceleration, and

deceleration counts across types. Picked some agents for time-series speed plots to show patterns.

For stats, used summary stats at the group level and some tests. T-tests for pairs, ANOVA for all groups together. Set alpha at 0.05. We might be oversimplifying the differences, but it points to variations between agent types.

The whole thing is in a Python script that's reproducible. Fixed parameters like the deceleration threshold and boundaries, no random stuff, so running it on the same data gives the same output every time. Some parts of the analysis feel a bit messy still, like how buffers affect approaches.